ЛР2. SAST: статический анализ кода с Semgrep

Время: 2 академических часа. **Что получится:** вы запустите Semgrep с двумя наборами правил, разберёте находки, отсеете ложные срабатывания, примените готовое исправление и убедитесь сканером, что оно сработало.

Код писать не нужно: исправления уже подготовлены.

Перед началом: выполнена ЛР1.

Коротко о главном

- **SAST** читает исходный код, не запуская программу. Быстро, показывает файл и строку.
- Сканер ищет то, на что есть **правила**. Нет правила — нет находки.
- **Ложное срабатывание (FP)** — сканер сообщил о проблеме, которой нет. **Пропуск (FN)** — проблема есть, сканер молчит. Бывает и то и другое.
- Каждую находку **разбирают**: настоящая → исправить; ложная → подавить с письменным обоснованием.
- Для конвейера важен **код завершения**: 0 — всё хорошо, не 0 — остановить изменение.
- ГОСТ Р 71207-2024 требует фиксировать версию анализатора, набор правил и решение по каждому срабатыванию. Вы делаете это в отчёте.

Термин	Значение
Правило (rule)	описание опасной конструкции, которую ищет сканер
Набор правил (ruleset)	группа правил; свой файл `.semgrep.yml` или готовый из реестра, например `p/default`
Severity	важность находки: ERROR, WARNING, INFO
SARIF	стандартный формат отчёта, который понимают GitHub и другие системы

Ход работы

Каталог проекта и окружение:

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
```

Шаг 1. Первый запуск

```
semgrep --version
semgrep scan --config .semgrep.yml --metrics=off .
```

- `--config .semgrep.yml` — правила из файла проекта (пять учебных правил);

- `--metrics=off` — не отправлять статистику разработчику сканера;
- `.` — проверять текущий каталог.

Что увидите: `Ran 5 rules on … files: 6 findings.`

Правило	Файл	Строка
`lab.github-token-literal`	`backend/config.py`	3
`lab.sql-injection-fstring-variable`	`backend/app.py`	37
`lab.sql-injection-fstring-direct`	`backend/app.py`	54
`lab.flask-debug-enabled`	`backend/app.py`	94
`lab.template-safe-filter`	`frontend/templates/comments.html`	14
`lab.template-safe-filter`	`frontend/templates/search.html`	9

Если `semgrep: command not found` — не активировано окружение `.venv`.

В отчёт: версия Semgrep и итоговая строка.

Шаг 2. Разобрать находки

Откройте каждый файл на указанной строке и заполните таблицу. Сопоставьте с атаками из ЛР1.

№	Правило	Файл:строка	Какой атаке из ЛР1 соответствует	Настоящая или ложная	Severity
1	`sql-injection-fstring-variable`	`app.py:37`	атака 1, вход без пароля	настоящая	ERROR
2					
…					

Вопрос: атаку 3 (чужой профиль) Semgrep не нашёл. Почему? Запишите ответ одним-двумя предложениями.

В отчёт: таблица и ответ.

Шаг 3. Запустить готовый набор правил

Свои пять правил — это мало. У Semgrep есть реестр готовых наборов. Запустите стандартный:

```
semgrep scan --config p/default --metrics=off .
```

Нужен интернет: правила скачиваются из реестра.

Что увидите: около 18 находок (в контрольном запуске — 18; правила в реестре обновляются, у вас может быть иначе).

Сравните два запуска и заполните таблицу:

Что	Свои правила	`p/default`
SQL-инъекция в `/login` и `/search`	найдено	
XSS через `safe` в шаблонах	найдено	
Отладочный режим	найдено	
Токен в `config.py`	найдено	
Сервер слушает на всех адресах (`0.0.0.0`)	—	
Контейнер работает от root (`Dockerfile`)	—	
Нет защиты форм от CSRF	—	

Выводы, которые нужно сделать самостоятельно:

1. Что нашёл только готовый набор?
2. Что готовый набор **пропустил**, хотя уязвимость настоящая и вы её эксплуатировали?
3. Одна и та же строка `app.py:54` попала в отчёт несколько раз под разными правилами. Это несколько уязвимостей или одна?

В отчёт: таблица и три ответа.

Шаг 4. Найти ложное срабатывание

В отчёте `p/default` есть правило `sqlalchemy-execute-raw-query` на строках `app.py:65` и `app.py:78`. Откройте эти строки:

```
connection.cursor().execute(
    f"INSERT INTO comments (author, content) VALUES ({mark}, {mark})",
    (author, content),
)
```

Здесь в текст запроса подставляется только `mark` — знак параметра (`?`), а не данные пользователя. Сами данные `(author, content)` передаются отдельно, как и положено. Инъекция невозможна. К тому же приложение вообще не использует библиотеку SQLAlchemy, для которой написано правило.

Это **ложное срабатывание**. Сравните со строкой 37, где в запрос подставляются `{username}` и `{password}` — там находка настоящая.

В отчёт: одно предложение — чем строка 65 отличается от строки 37.

Шаг 5. Управлять выводом сканера

Фильтр по важности — показать только ERROR:

```
semgrep scan --config .semgrep.yml --metrics=off --severity ERROR .
```

Что увидите: `3 findings` вместо шести.

Отчёты в файл — JSON для отчёта, SARIF для загрузки в другие системы:

```
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-befo
re.json --sarif-output reports/sast-before.sarif .
ls reports
```

Код завершения — главное для конвейера:

```
semgrep scan --config .semgrep.yml --metrics=off -q . ; echo "Код: $?"
semgrep scan --config .semgrep.yml --metrics=off -q --error . ; echo "Код: $?"
```

Что увидите: первая команда — `Код: 0`, вторая — `Код: 1`.

Без `--error` Semgrep возвращает 0, даже когда находки есть: конвейер посчитал бы проверку пройденной. С `--error` любая находка даёт код 1 и останавливает конвейер. Запомните этот параметр — он понадобится в ЛР5.

В отчёт: число находок с фильтром ERROR, оба кода завершения и объяснение разницы.

Шаг 6. Применить готовое исправление

Исправленные файлы лежат в материалах курса. Скопируйте их в проект:

```
cp -r ../secure-development-labs/solutions/lab2/. .
git status --short
```

Что увидите: четыре изменённых файла — `app.py`, два шаблона и тесты.

Посмотрите, что именно изменилось в коде:

```
git diff backend/app.py
```

Строки с минусом удалены, с плюсом — добавлены. Суть исправлений:

Было	Стало	Почему это защита
данные пользователя вклеивались в текст SQL	текст запроса и данные передаются раздельно	база больше не может принять данные за команду
`{{ query|safe }}` в шаблонах	`{{ query }}`	текст пользователя выводится как текст, теги не исполняются
`debug=True`	`debug=False`	отключена отладочная консоль

Специалисту по безопасности не обязательно писать такой код, но нужно уметь прочитать изменение и понять, устранена ли причина.

Шаг 7. Проверить исправление

Три независимые проверки.

Тесты. Вместе с исправлением пришли тесты, которые пытаются повторить атаки:

```
pytest -q
```

Что увидите: `7 passed`.

Сканер.

```
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-after.json .
```

Что увидите: `1 finding` — остался только токен в `config.py`. Им займётесь в ЛР3.

Вручную. Запустите приложение (`python -m backend.app`) и повторите атаки 1 и 2 из ЛР1.

Что увидите: вход с `admin' --` даёт «Неверные учётные данные»; комментарий с тегами отображается текстом, окно не всплывает.

Остановите сервер (`Ctrl+C`).

В отчёт: вывод `pytest`, итог Semgrep, снимок экрана неудавшейся атаки.

Шаг 8. Подавить ложное срабатывание

Запустите готовый набор ещё раз:

```
semgrep scan --config p/default --metrics=off .
```

Что увидите: около 8 находок вместо 18. Четыре из них — то самое ложное срабатывание `sqlalchemy-execute-raw-query`: исправленный код использует параметры, но правило этого не понимает.

Если ложные срабатывания оставить, отчёт всегда будет «красным», и на него перестанут смотреть. Отключите это правило для проекта:

```
semgrep scan --config p/default --metrics=off --exclude-rule python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query .
```

Что увидите: около 4 находок. Разберите оставшиеся:

Правило	Настоящая?	Что с ней делать
`missing-user` (Dockerfile)		исправляется в ЛР4
`avoid_app_run_with_bad_host`		
`django-no-csrf-token` (2 шт.)		

Правила подавления:

- подавляют **конкретное** правило или строку, а не «всё, что мешает»;
- у подавления есть **письменное обоснование** и автор;
- настоящую уязвимость не подавляют — её исправляют или официально принимают риск.

Подавить одну строку можно комментарием в коде: `# nosemgrep: идентификатор-правила`.

В отчёт: числа находок 18 → 8 → 4, таблица и обоснование подавления одним-двумя предложениями.

Шаг 9. Как это встраивается в CI

В ЛР5 в конвейере будет ровно та команда, которую вы уже выполняли:

```
- name: Run Semgrep
  run: semgrep scan --config .semgrep.yml --metrics=off --error --json-output re
ports/semgrep.json .
```

Три вещи, которые делают её пригодной для конвейера: закреплённая версия сканера (в `requirements-dev.txt`), свой проверенный набор правил без ложных срабатываний и `--error` для ненулевого кода завершения.

Вопрос для отчёта: почему в конвейер поставлены свои правила, а не `p/default`? Подсказка: вспомните шаг 8 и то, что правила в реестре меняются без вашего ведома.

Шаг 10. Сохранить результат

```
git add -A
git commit -m "lab2: fix SQL injection, XSS and debug mode"
git log --oneline
```

Что увидите: два коммита. Папка `reports` в коммит не попадает.

Отчёт

1. Версия Semgrep, итог первого запуска.
2. Таблица шести находок, ответ про атаку 3.
3. Сравнение своих правил и `p/default`, три ответа.
4. Разбор ложного срабатывания.
5. Фильтр по важности, коды завершения.
6. Результаты трёх проверок после исправления.
7. Числа 18 → 8 → 4, таблица остатка, обоснование подавления.
8. Ответ на вопрос шага 9.

Контрольные вопросы

1. Чем ложное срабатывание отличается от пропуска? Приведите по примеру из этой работы.
2. Сканер показал 0 находок. Что из этого следует, а что нет?
3. Зачем нужен параметр `--error`?
4. Когда находку можно подавить, а когда нельзя?
5. Почему одного набора правил недостаточно?

Оценивание

Критерий	Баллы
Запуск и разбор шести находок	20
Сравнение наборов правил	20
Ложное срабатывание: найдено, объяснено, подавлено	20

Критерий	Баллы
Управление выводом и кодом завершения	15
Проверка исправления тремя способами	15
Контрольные вопросы	10
Итого	**100**